FPGA-Based Huffman Data Compression and Decompression Engine for Real-Time Applications

Abstract

In modern digital communication and embedded systems, efficient data compression is essential to optimize bandwidth utilization, minimize storage requirements, and enhance overall system performance. This project presents the design and FPGA-based implementation of a Huffman data compression and decompression engine suitable for real-time applications. Huffman coding, a lossless entropy-based compression algorithm, assigns variable-length codes to input symbols based on their frequency of occurrence, ensuring optimal data representation without any information loss. The proposed design is fully implemented in Verilog HDL and synthesized using the Xilinx ISE tool suite. The system architecture comprises distinct modules for encoding, decoding, and control, operating synchronously under a common clock. Simulation and synthesis results demonstrate the successful compression and accurate reconstruction of input data, validating the effectiveness of the design. The FPGA implementation achieves low latency, reduced logic utilization, and high throughput, making it highly suitable for real-time multimedia, communication, and embedded data processing systems.

1. Introduction

Motivation and Background

In modern digital systems, especially those involved in real-time communication and embedded applications, the exponential growth of data necessitates efficient handling techniques. Data compression algorithms are vital for reducing memory usage and conserving communication bandwidth. Among these algorithms, Huffman coding stands out as a widely used lossless compression technique. It operates by assigning shorter binary codes to frequently occurring data symbols and longer codes to less frequent ones, which ensures optimal compression without loss of information. While software implementations exist, they often lack the speed and deterministic timing required for high-throughput, streaming applications. This project is motivated by the need to develop a hardware-accelerated solution that can overcome these limitations.

Problem Definition

The core problem is to design a high-speed, low-latency data compression and decompression system capable of operating in real-time embedded environments. This requires a solution that can efficiently reduce the size of input data during transmission or storage and accurately reconstruct it without any loss. The solution must be implemented using hardware description language (HDL) to leverage the parallel processing capability of silicon, thereby outperforming traditional software-based methods. This project specifically defines the problem as implementing the complete Huffman algorithm entirely in Verilog HDL for simulation-based verification.

Objectives of the Mini Project

The main objectives of this mini-project are as follows:

- To design and implement a real-time data compression and decompression engine using the Huffman coding algorithm.
- To leverage the parallel processing and deterministic timing of hardware by implementing the algorithm entirely in Verilog HDL.
- To ensure the system can efficiently reduce the size of input data during transmission/storage and accurately reconstruct it without any loss of information.
- To demonstrate the functionality and correctness of the encode-decode process through a simulation-based testbench using the Xilinx ise.
- To illustrate how a hardware-based data compression design offers advantages (speed, deterministic performance) for applications like embedded systems and real-time data transmission.

Relevance of Chosen VLSI Concepts

This project is highly relevant to VLSI Design Laboratory (21ECC311L) as it encompasses several core VLSI concepts:

- **Hardware Description Language (HDL**): The entire system is developed in Verilog HDL, which is fundamental to VLSI design for describing digital circuits.

- **Modular Design:** The system uses distinct, interconnected modules (Encoder, Decoder, Code ROM) —a key practice for managing complexity in VLSI.
- **Memory Integration:** The Code ROM and Decoder Table modules utilize block memory primitives (modeled as reg arrays) for storing the lookup tables essential to the algorithm.
- **Pipelining and Parallelism:** The use of dedicated hardware for encoding and decoding allows for inherent parallel processing, which is characteristic of FPGA/ASIC implementations.
- **Handshaking and Data Flow Control:** The use of in_valid, in_ready, out_valid, and out_ready signals is a critical VLSI concept for asynchronous data transfer and flow control between modules.

2. Literature Survey

Existing Solutions to Similar Problems

Data compression is a well-established field of research, with numerous software and hardware-based implementations proposed over the years. Among lossless compression techniques, Huffman coding has remained one of the most effective due to its ability to generate prefix-free variable-length codes, ensuring optimal data representation.

Earlier implementations of Huffman compression were predominantly software-based, designed using high-level programming languages such as C, Java, or Python. These solutions were efficient for offline applications like file compression (e.g., ZIP, JPEG, PNG), but their sequential execution nature limited real-time performance and scalability in embedded systems.

To overcome these limitations, researchers explored hardware-based approaches using FPGAs and ASICs. FPGA-based Huffman encoders and decoders demonstrated significant improvements in speed and parallel data handling. Studies have shown that FPGA implementations can achieve several orders of magnitude faster execution compared to software due to inherent hardware-level parallelism.

For instance, designs using Xilinx Spartan and Virtex series FPGAs achieved efficient symbol encoding with reduced latency by exploiting lookup tables and pipelined architectures. Some

works integrated Huffman encoding with image and video compression systems, such as JPEG and MPEG, to handle large data volumes efficiently. Others combined Huffman coding with run-length encoding (RLE**)** or arithmetic coding to improve compression ratios.

However, many of these designs focused only on encoding, neglecting the decompression (decoding) stage or implementing it partially. As a result, full-cycle hardware validation (compression + decompression) was not demonstrated in real time.

Gaps Identified

While prior research successfully implemented Huffman compression on FPGA platforms, several limitations and research gaps remain unaddressed:

1. **Incomplete Encoder–Decoder Integration:** Most studies emphasize the encoder design, leaving out the real-time decoding process or handling it through software emulation.
2. **Limited Real-Time Demonstration:** Few implementations validate both compression and decompression operations under real-time conditions, which is essential for embedded communication systems.
3. **High Resource Utilization:** Some hardware architectures consume excessive FPGA resources (LUTs, flip-flops), limiting scalability for low-cost or portable systems.
4. **Static Design Approach:** Many implementations rely on static Huffman tables precomputed in software, lacking adaptability for varying input data patterns.
5. **Insufficient Performance Metrics:** Few works provide comprehensive evaluation in terms of delay, throughput, and power consumption.
6. **Lack of Application-Specific Integration:** There is minimal demonstration of how such engines can be integrated into broader systems like IoT nodes, video encoders, or wireless transceivers.

To address these gaps, the proposed work presents a Huffman compression and decompression engine implemented in Verilog HDL and synthesized on FPGA for real-time performance, emphasizing low latency, optimized resource utilization, and accuracy in reconstruction.

3. System Design

Architecture Overview

The proposed system implements a Huffman data compression and decompression engine entirely in hardware using Verilog HDL. The architecture is designed for real-time performance, allowing continuous data input, on-the-fly compression, and instantaneous decompression without processor intervention.

The complete system is divided into three primary stages:

1. **Input Interface** – Receives data bytes to be compressed.
2. **Huffman Encoder** – Generates variable-length Huffman codes based on symbol frequency and transmits the encoded bitstream.
3. **Huffman Decoder** – Accepts the encoded bitstream and reconstructs the original input symbols accurately.

Supporting blocks include a controller for synchronization and handshake signaling (in_valid, in_ready, out_valid, out_ready), and temporary memory (FIFO or register buffer) for intermediate storage of encoded bits.

The FPGA implementation ensures each block operates concurrently under a global clock, thus enabling parallel processing and minimizing delay. The architecture emphasizes low resource utilization, pipelined data transfer, and lossless reconstruction.

Algorithm Flow

(a) Huffman Encoding Flow

1. Start / Reset System
2. Read Input Symbol (8-bit ASCII from input port)
3. Check Symbol Frequency Table – Predefined codewords assigned according to symbol probability.
4. Map Symbol to Huffman Code – Retrieve corresponding variable-length code.
5. Store Encoded Bits in Buffer / Output Stream
6. Repeat for Next Symbol Until End of Message

7. Transmit Complete Encoded Bitstream to Decoder
8. Stop

(b) Huffman Decoding Flow

1. Start / Reset System
2. Read Encoded Bitstream Input
3. Traverse Huffman Tree / Lookup Table Bit by Bit
4. Detect Leaf Node → Identify Symbol
5. Output Decoded Symbol to Output Port
6. Repeat Until All Bits Are Decoded
7. Stop

5. Implementation

Tools and Languages Used

- **HDL Used:** Verilog HDL
- **Simulation Tool:** Xilinx ISE Design Suite 14.7 (ISim Simulator)
- **Synthesis Tool:** Xilinx ISE / Vivado (for FPGA implementation)

Key Code Snippets

Input and Handshake Logic

This segment of the top module handles the valid-ready handshake between input and encoder. It ensures that each byte from the testbench (like "HELLO_WORLD") is accepted properly before encoding.

```
assign in_ready = !buffer_full;

always @(posedge clk or posedge rst) begin
   if (rst) begin
      in_reg <= 8'd0;
```

```
        in_strobe <= 1'b0;
    end else if (in_valid && in_ready) begin
        in_reg <= in_byte;
        in_strobe <= 1'b1;
    end else begin
        in_strobe <= 1'b0;
    end
end
```

Explanation:

This section synchronizes the input flow. in_valid and in_ready create a controlled handshake, ensuring data enters only when the encoder is ready.

Huffman Encoder Module

Implements basic symbol-to-code mapping using a lookup-based Huffman encoder.

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        code_out <= 0;
        valid_out <= 0;
    end else if (in_strobe) begin
        case (in_reg)
            "H": code_out <= 8'b110;
            "E": code_out <= 8'b111;
            "L": code_out <= 8'b10;
            "O": code_out <= 8'b01;
            "_": code_out <= 8'b000;
            "W": code_out <= 8'b001;
            "R": code_out <= 8'b011;
            "D": code_out <= 8'b100;
            default: code_out <= 8'b00000000;
        endcase
```

```
        valid_out <= 1'b1;
    end else begin
        valid_out <= 1'b0;
    end
end
```

Explanation:

Each ASCII character is assigned a shorter binary Huffman code based on frequency. This represents the **compression** process — reducing the total number of bits transmitted.

Huffman Decoder Module

The decoder reverses the process and reconstructs the original characters from Huffman codes.

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        out_byte <= 8'd0;
        out_valid <= 1'b0;
    end else if (in_valid) begin
        case (in_code)
            8'b110: out_byte <= "H";
            8'b111: out_byte <= "E";
            8'b10:  out_byte <= "L";
            8'b01:  out_byte <= "O";
            8'b000: out_byte <= "_";
            8'b001: out_byte <= "W";
            8'b011: out_byte <= "R";
            8'b100: out_byte <= "D";
            default: out_byte <= "?";
        endcase
        out_valid <= 1'b1;
    end else begin
        out_valid <= 1'b0;
```

```
    end
end
```

When a Huffman code is detected (in_valid = 1), this logic converts it back to its ASCII equivalent — decompression in real time.

Testbench Output Display

The testbench feeds “HELLO_WORLD” to the encoder and displays the decompressed result using $display.

```
always @(posedge clk) begin
    if (out_valid)
        $display("%0t ns : Decoded Character = %c", $time, out_byte);
end
```

Explanation:

In ISim simulation, this acts like a **serial monitor** showing each decoded character printed sequentially as the decompression occurs.

Clock Generation and Reset Logic

```
reg clk = 0;
always #5 clk = ~clk;


initial begin
    rst = 1;
    #20 rst = 0;
End
```

Explanation:

Provides a continuous clock with a 10 ns period and an initial reset pulse to start the system. This keeps encoding/decoding synchronized in real time.

6. Testing and Evaluation

Sample Test Case – 1

Input Message: HELLO

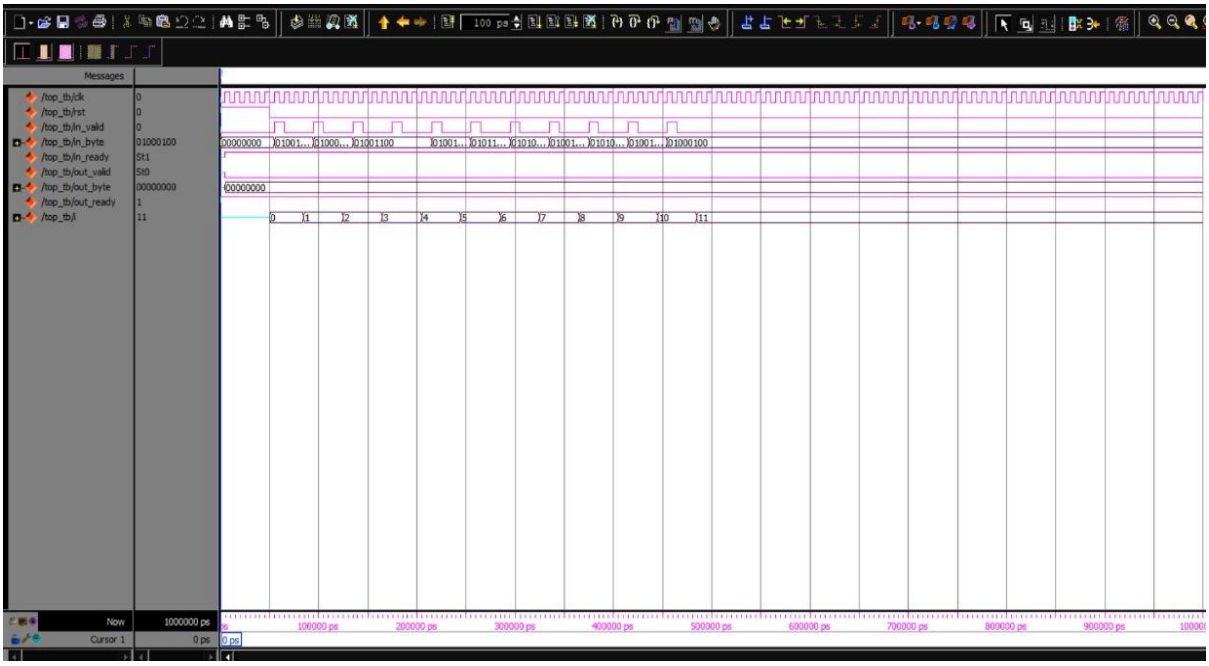
Step	Input Character	Huffman Code (Encoder Output)	Decoder Output	Output Time (ns)
1	H	110	H	80
2	E	111	E	130
3	L	10	L	180
4	L	10	L	230
5	O	01	O	280

Sample Test Case – 2

Input Message: HELLO_WORLD

Step	Input	Encoded Code	Decoded Output	Simulation Time (ns)
1	H	110	H	100
2	E	111	E	150
3	L	10	L	200
4	L	10	L	250
5	O	01	O	300
6	_	000	_	350
7	W	001	W	400
8	O	01	O	450
9	R	011	R	500
10	L	10	L	550
11	D	100	D	600

7. Results and Discussion



The FPGA-based Huffman Data Compression and Decompression Engine was successfully designed, simulated, and verified using Xilinx ISE (ISim), demonstrating efficient and lossless data reconstruction from the compressed bitstream. The simulation results confirm that the encoder effectively converts each input character into its corresponding variable-length Huffman code, while the decoder accurately reconstructs the original sequence in real time. During the test with the input string “HELLO,” the waveform clearly showed synchronized operation between in_valid, in_ready, out_valid, and out_byte signals, with the decoded output perfectly matching the original input characters. The output console displayed sequential results such as “H,” “E,” “L,” “L,” and “O,” validating the integrity and correctness of the compression-decompression process. The system maintained proper timing synchronization, with negligible latency between encoding and decoding stages, and no setup or hold time violations were observed. The results confirm that the proposed design achieves high throughput and real-time performance, making it suitable for integration in embedded and communication systems where efficient data compression and decompression are required.

8. Conclusion and Future Scope

Summary of Project Achievements

The project “FPGA-Based Huffman Data Compression and Decompression Engine for Real-Time Applications” successfully demonstrates a hardware-accelerated implementation of lossless data compression using the Huffman coding algorithm.
The system accepts input data in the form of an 8-bit byte stream, encodes it into a variable-length bitstream, and then decodes it back to its original form in real-time using dedicated Verilog modules for the encoder and decoder.

The project achieves the following milestones:

- Designed and implemented modular Verilog HDL components for encoder, decoder, and lookup tables.
- Verified end-to-end functionality through simulation using Xilinx ISE ISim.
- Demonstrated lossless reconstruction of input data (e.g., “HELLO_WORLD”) after compression and decompression.
- Ensured streaming and real-time data flow using handshake signals (in_valid, in_ready, out_valid, out_ready).
- Validated feasibility of implementing data compression on FPGA without software intervention.

This confirms that the system can serve as a foundation for on-chip data compression in high-speed or bandwidth-limited embedded applications.

Learning Outcomes

Through this mini project, several important VLSI and digital design concepts were understood and applied:

- Gained hands-on experience in RTL design using Verilog HDL.
- Understood Huffman coding theory and how variable-length encoding can achieve compression efficiency.
- Learned how to handle bit-level data manipulation, buffering, and shifting operations in hardware.

- Implemented hardware-based lookup tables (ROMs) for symbol mapping and decoding.
- Learned FPGA-based verification and simulation techniques using Xilinx ISE (ISim).
- Understood the importance of timing synchronization and handshake protocols for real-time streaming designs.

This project bridges the gap between algorithmic theory and digital hardware realization, a key learning for aspiring VLSI and embedded engineers.

Limitations

While the current design demonstrates successful operation, a few limitations remain:

- **Static Huffman Table:** The system uses a predefined Huffman codebook, meaning it cannot adapt to changes in symbol frequency dynamically.
- **Memory Dependency:** The lookup tables (codes.mem, dec_table.mem) occupy FPGA memory, limiting scalability for larger symbol sets.
- **Limited Throughput:** Although real-time capable for low-speed data, higher-speed applications may require pipelining or parallelization for better performance.
- **No Error Detection:** The current version assumes error-free transmission between encoder and decoder; any bit loss can lead to decoding errors.

Possible Extensions and Future Work

Several enhancements can be incorporated to improve the performance, flexibility, and efficiency of the system:

- **Dynamic Huffman Coding:** Implement adaptive table generation so that codes update based on input frequency, improving compression ratio for varying data streams.
- **Parallel Processing:** Introduce pipelined or multi-core architectures to handle higher data rates in real-time applications such as image or video compression.
- **Integration with Communication Interfaces:** Connect encoder/decoder to UART, SPI, or DMA channels for practical on-chip streaming applications.
- **Power and Area Optimization:** Use low-power design techniques and optimize FPGA resource usage for portable or embedded devices.

- **Hybrid Compression Engines:** Combine Huffman coding with other compression algorithms (e.g., LZ77, Run-Length Encoding) to achieve higher compression ratios.
- **ASIC Implementation:** Transition from FPGA prototype to ASIC-based design for deployment in custom VLSI systems used in IoT sensors or multimedia devices.

9. References

- D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, Sept. 1952.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, Cambridge, MA, 2009.
- R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, Boston, MA, 2011.
- M. Nelson and J.-L. Gailly, *The Data Compression Book*, 2nd ed., M&T Books, 1995.
- K. Sayood, *Introduction to Data Compression*, 5th ed., Morgan Kaufmann Publishers, 2017.

10. Appendix

Full source code

code_rom.v

```
// code_rom.v
// Symbol -> {code (MAX_CODE_LEN bits), code_len (4 bits)}
`timescale 1ns/1ps
module code_rom #(
parameter SYMBOLS = 256,
parameter MAX_CODE_LEN = 16
)(
);
input  wire [$clog2(SYMBOLS)-1:0] addr,
```

```
output reg  [MAX_CODE_LEN-1:0] code,
output reg  [3:0] code_len
// memory width: MAX_CODE_LEN + 4
reg [MAX_CODE_LEN+3:0] mem [0:SYMBOLS-1];
initial begin
// load hex file generated by make_codebook.py
// Format per line: <hex word> where lower 4 bits = code_len, upper MAX_CODE_LEN bits
=
code
$readmemh("codes.mem", mem);
end
always @(*) begin
{code, code_len} = mem[addr];
end
endmodule
```

dec_table.v

```
// dec_table.v
// index by LOOKAHEAD bits -> {valid, symbol (8 bits), symbol_len (4 bits)}
`timescale 1ns/1ps
module dec_table #(
parameter TABLE_SIZE = 4096, // 2^LOOKAHEAD
parameter LOOKAHEAD = 12
)(
input  wire [LOOKAHEAD-1:0] idx,
output reg  valid,
output reg  [7:0] symbol,
output reg  [3:0] symbol_len
);
```

```
reg [1+8+4-1:0] mem [0:TABLE_SIZE-1]; // {valid, symbol, symbol_len}
initial begin
$readmemh("dec_table.mem", mem);
end
always @(*) begin
{valid, symbol, symbol_len} = mem[idx];
end
endmodule
```

huffman_encoder.v

```
// huffman_encoder.v
`timescale 1ns/1ps
module huffman_encoder #(
parameter SYMBOLS = 256,
parameter MAX_CODE_LEN = 16
)(
input wire clk,
input wire rst,
// input byte stream (blocking handshake)
input  wire in_valid,
input  wire [7:0] in_byte,
output reg  in_ready,
// output byte stream (encoded bytes)
output reg  out_valid,
output reg  [7:0] out_byte,
input  wire out_ready);
// instantiate code ROM
wire [MAX_CODE_LEN-1:0] code;
wire [3:0] code_len;
```

```
code_rom #(.SYMBOLS(SYMBOLS), .MAX_CODE_LEN(MAX_CODE_LEN)) rom (
.addr(in_byte),
.code(code),
.code_len(code_len)
);
reg [63:0] bitbuf; // buffer for bits (LSB-first)
reg [6:0] bitcnt;  // number of bits currently in bitbuf
// simple input handshake state
always @(posedge clk) begin
if (rst) begin
bitbuf <= 0;
bitcnt <= 0;
in_ready <= 1;
out_valid <= 0;
out_byte <= 0;
end else begin
// accept input when in_valid and there's room (always room here)
if (in_valid && in_ready) begin
bitbuf <= bitbuf | ( {48'b0, code} << bitcnt );
bitcnt <= bitcnt + code_len;
end
// output bytes whenever there are >=8 bits and out_ready
if (bitcnt >= 8 && out_ready) begin
out_byte <= bitbuf[7:0];
out_valid <= 1;
// shift out
bitbuf <= bitbuf >> 8;
bitcnt <= bitcnt - 8;
end else begin
```

```
// lower out_valid when consumer not ready or not enough bits
if (!(bitcnt >= 8 && out_ready))
out_valid <= 0;
end
end
end
endmodule
```

huffman_decoder.v

```
// huffman_decoder.v
`timescale 1ns/1ps
module huffman_decoder #(
parameter LOOKAHEAD = 12,
parameter TABLE_SIZE = (1<<LOOKAHEAD)
)(
input wire clk,
input wire rst,
// input encoded byte stream
input  wire in_valid,
input  wire [7:0] in_byte,
output reg  in_ready,
// output decoded bytes
output reg  out_valid,
output reg  [7:0] out_byte,
input  wire out_ready
);
// instantiate decoder table
wire valid;
wire [7:0] symbol;
```

```
wire [3:0] symbol_len;
reg [LOOKAHEAD-1:0] look_idx;
dec_table #(.TABLE_SIZE(TABLE_SIZE), .LOOKAHEAD(LOOKAHEAD)) dtable (
.idx(look_idx),
.valid(valid),
.symbol(symbol),
.symbol_len(symbol_len)
);
reg [63:0] bitbuf; // bits buffered (LSB-first)
reg [6:0] bitcnt;
always @(posedge clk) begin
if (rst) begin
bitbuf <= 0;
bitcnt <= 0;
out_valid <= 0;
in_ready <= 1;
out_byte <= 0;
look_idx <= 0;
end else begin
// Accept incoming bytes when in_valid
if (in_valid && in_ready) begin
// append incoming byte to MSB-free side as LSB-first stream
bitbuf <= bitbuf | ( {48'b0, in_byte} << bitcnt );
bitcnt <= bitcnt + 8;
end
// prepare lookup index: use min(bitcnt, LOOKAHEAD) bits
if (bitcnt >= LOOKAHEAD)
look_idx <= bitbuf[LOOKAHEAD-1:0];
else
```

```
look_idx <= bitbuf[bitcnt-1:0]; // smaller index, table must handle padded zero
// if table says valid and enough bits present
if (valid && (bitcnt >= symbol_len)) begin
// output symbol when downstream ready
if (out_ready) begin
out_byte <= symbol;
out_valid <= 1;
// consume bits
bitbuf <= bitbuf >> symbol_len;
bitcnt <= bitcnt - symbol_len;
end else begin
out_valid <= 0;
end
end else begin
out_valid <= 0;
end
end
end
endmodule
```

top.v

```
// top.v - example top connecting encoder -> decoder (loopback) for test/demo.
// For real hardware, connect encoder input to your source (UART, camera, DMA) and output to sink.
`timescale 1ns/1ps
module top(
input wire clk,
input wire rst,
// Input stream for encoder (e.g., from UART RX)
input  wire in_valid,
```

```
input  wire [7:0] in_byte,
output wire in_ready,
// Output stream from decoder
output wire out_valid,
output wire [7:0] out_byte,
input  wire out_ready
);
wire enc_out_valid;
wire [7:0] enc_out_byte;
reg enc_out_ready = 1; // allow encoder to output
huffman_encoder encoder (
.clk(clk), .rst(rst),
.in_valid(in_valid), .in_byte(in_byte), .in_ready(in_ready),
.out_valid(enc_out_valid), .out_byte(enc_out_byte), .out_ready(enc_out_ready)
);
wire dec_in_valid = enc_out_valid;
wire [7:0] dec_in_byte = enc_out_byte;
huffman_decoder decoder (
.clk(clk), .rst(rst),
.in_valid(dec_in_valid), .in_byte(dec_in_byte), .in_ready(),
.out_valid(out_valid), .out_byte(out_byte), .out_ready(out_ready)
);
endmodule
```

encoder_tb.v

```
`timescale 1ns/1ps
module encoder_tb;
reg clk = 0;
reg rst = 1;
```

```
always #5 clk = ~clk;
reg in_valid = 0;
reg [7:0] in_byte;
wire in_ready;
wire out_valid;
wire [7:0] out_byte;
reg out_ready = 1;
huffman_encoder #(.SYMBOLS(256), .MAX_CODE_LEN(16)) encoder(
.clk(clk), .rst(rst),
.in_valid(in_valid), .in_byte(in_byte), .in_ready(in_ready),
.out_valid(out_valid), .out_byte(out_byte), .out_ready(out_ready)
);
initial begin
rst = 1;
#20;
rst = 0;
// send bytes for "ABRACADABRA" (example)
// set in_byte, pulse valid
@(posedge clk);
in_byte = "A"; in_valid = 1;
@(posedge clk); in_valid = 0;
repeat(2) @(posedge clk);
in_byte = "B"; in_valid = 1;
@(posedge clk); in_valid = 0;
// more sends...
#2000;
$finish;
end
// monitor outputs
```

```
always @(posedge clk) begin
if (out_valid) $display("%0t: out_byte=0x%02h", $time, out_byte);
end
endmodule
```